

TRAE CN 实战：从 API 文档到可复用服务的自动化探索

TRAE.ai 2025年12月14日 11:00 北京



本文作者：茉卷，TRAE 开发者用户

在日常开发中，我们经常会遇到这样的场景：产品经理给你一份 API 文档或者一段 curl 示例，然后你需要：

- 写请求封装代码（Service 层）
- 搭建临时测试页面（UI 层）
- 处理各种边界情况和错误展示

这些工作本质上很机械，却又是开发流程中无法跳过的环节。每次都要手动写一遍，既耗时又容易出错。

如果有一种方式，能让 AI 帮我们完成这些重复劳动，把“一次性的 API 调用”变成“工具箱里的通用积木”，会怎么样？

本文将通过一个具体案例来展示这个过程：

- 在扣子（Coze）里搭一个 **“飞书群通知”工作流**；
- 然后用 TRAE CN SOLO：
 - 读懂这份 workflow API 文档；
 - 自动生成一个 **可复用的工作流服务**（`workflowService`）；
 - 顺手生成一个 **React UI 测试界面**，用来给飞书机器人发消息。

扣子 + 飞书机器人 只是业务场景，真正的主角是 TRAE：

让我们看看 TRAE 是如何把这个过程从“手工作坊”变成“自动化生产线”的。



场景搭建：扣子 + 飞书机器人

1.1 扣子：快速生产“后端服务”的工厂

扣子（Coze）是一个低代码智能体平台，它的核心能力是：通过拖拽节点和简单配置，快速搭建智能工作流，并将其封装成 HTTP API 对外开放。

你可以把它理解成一个“后端服务工厂”——不需要写代码，就能生产出可用的 API 接口。

在本文的案例中，我们要搭建的工作流非常简单：

- **输入**：一个字符串 *input*（要发送到飞书群里的内容）；
- **中间步骤**：调用飞书自定义机器人 webhook；
- **输出**：把飞书调用的结果放在 *content* 字段里返回。

扣子会把整个 workflow 暴露成一个 HTTP 接口，我们稍后会交给 TRAE：

```
1 curl -X POST 'https://api.coze.cn/v1/workflow/stream_run' \  
2   -H "Authorization: Bearer YOUR_TOKEN" \  
3   -H "Content-Type: application/json" \  
4   -d '{  
5     "workflow_id": "7577226183415316506",  
6     "parameters": {  
7       "input": "你好！"  
8     }  
9   }'
```

其中的关键参数包括：

- **Bearer**：扣子平台的访问 Token（字符串，需保密）；
- **workflow_id**：工作流 URL 中 *workflow_id*= 后面的那段字符串；
- **input**：我们要发送到飞书群的消息内容。

工作流执行成功后，会返回类似这样的结构：

```
1 {  
2   "usage": {  
3     "token_count": 0,  
4     "output_count": 0,  
5     "input_count": 0  
6   },  
7   "node_is_finish": true,
```

```
8 "content": "{\\"code\\":0,\\"log_id\\":\\"20251127111551A90DF8D30A0605075D\\",\n9 "content_type": "text",\n10 "node_id": 900001,\n11 "node_execute_uuid": "",\n12 "node_seq_id": 0,\n13 "node_title": "End",\n14 "node_type": "End"\n15 }
```

我们真正关心的是 *content* 里的 JSON 字符串：

`{"code":0,"msg":"success",...}`，它告诉我们这次飞书推送是否成功。

1.2 飞书群与自定义机器人

为了让扣子工作流有地方“发消息”，我们需要在飞书中完成以下准备工作：

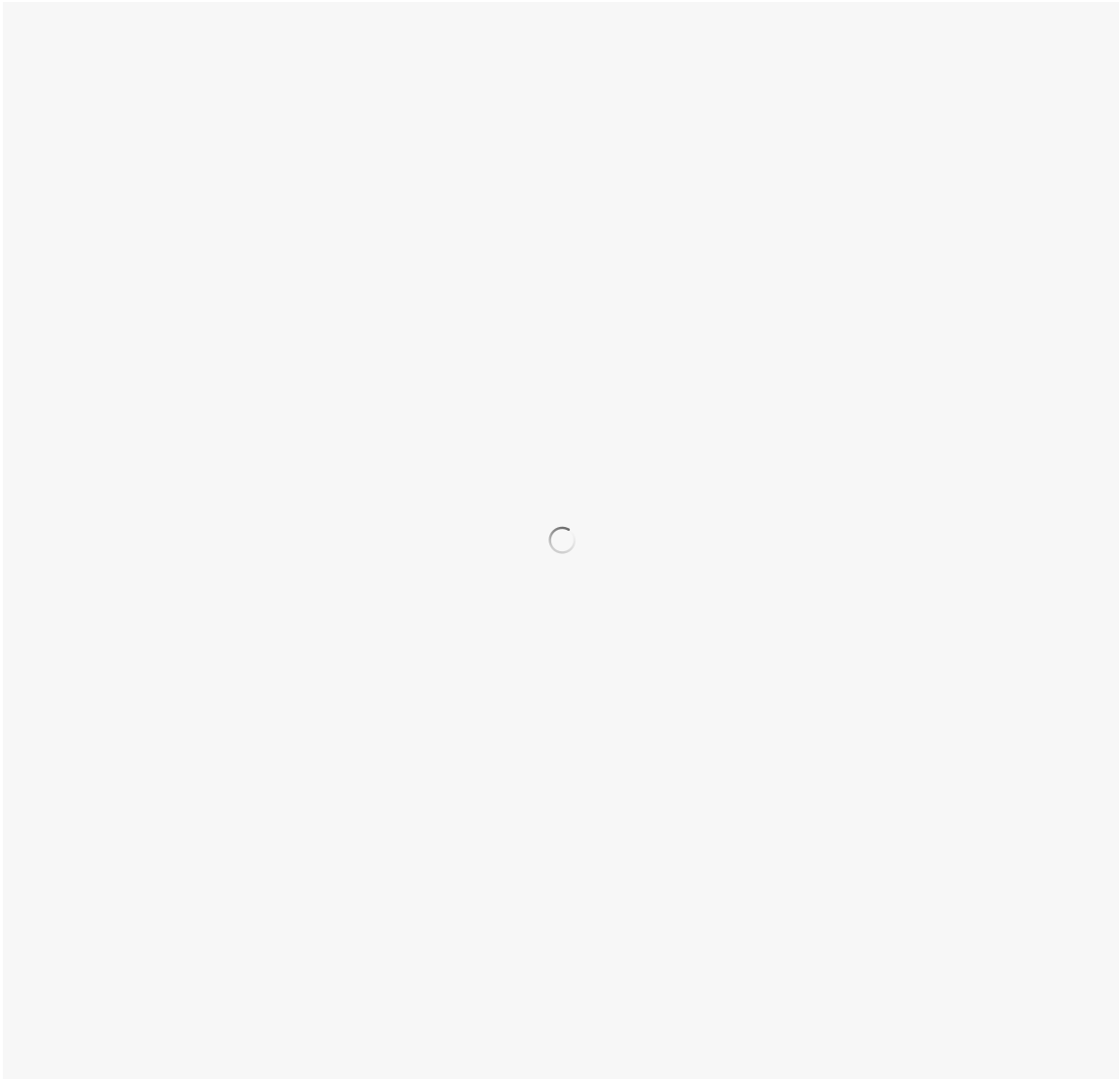
- 一个飞书群；
- 群里配置好一个 **自定义机器人**，拿到对应的 **Webhook 地址**。

操作步骤：

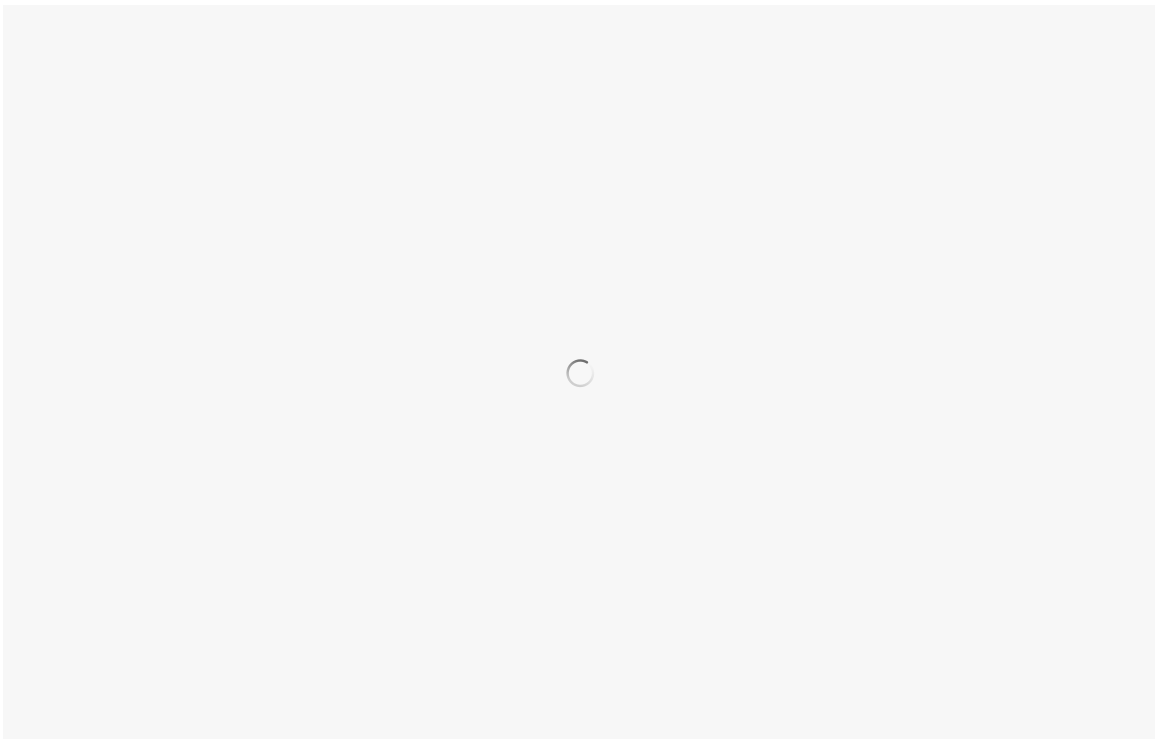
- 在飞书左上角点击 **「+」→ 创建群组**；



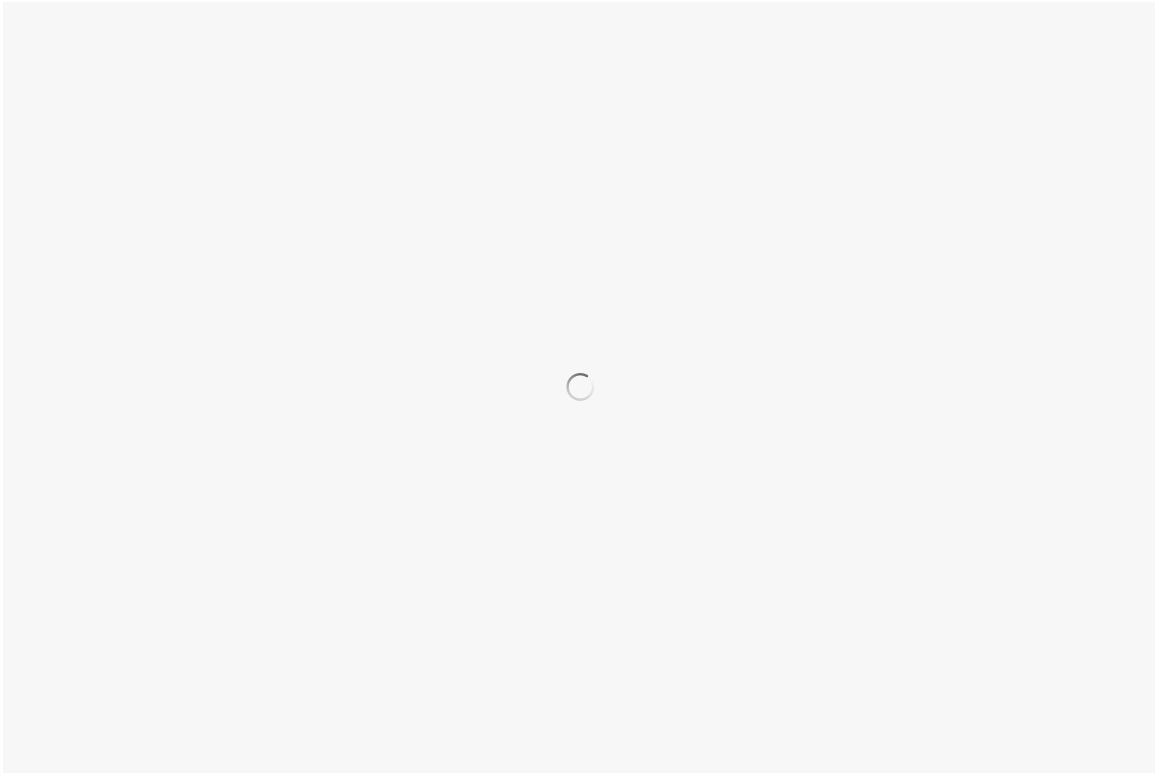
- 群模式选择「对话」，给群起一个名字（如“测试飞书机器人”）；



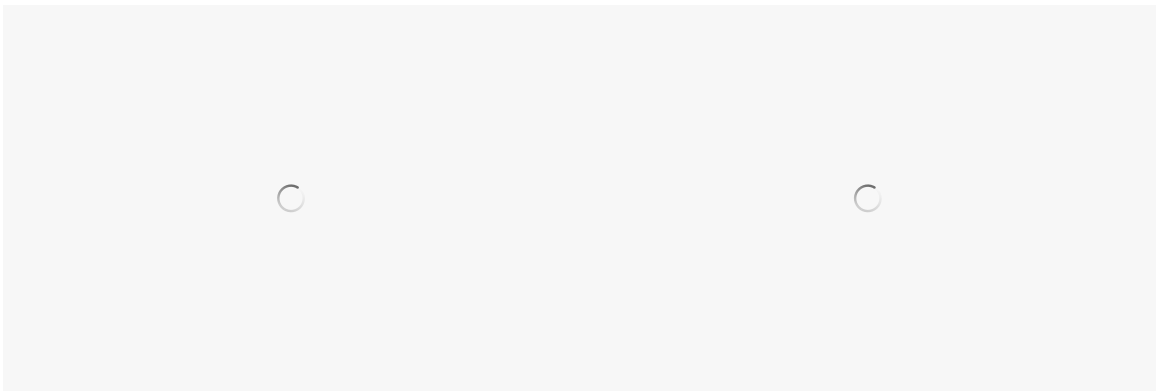
- 进入群设置 → 找到「群机器人」；



- 添加机器人 → 选择「自定义机器人」；

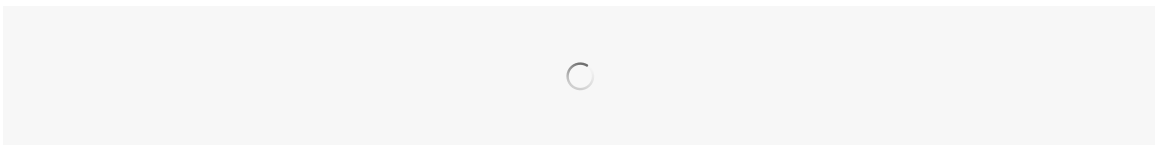


- 选一个头像、名字，点击添加，复制 Webhook 地址。



这个 Webhook 会被配置在扣子的 workflow 里，用于把消息真正发进群。

至此，我们的“消息接收端”就准备好了。接下来要做的，就是在扣子中把这个 Webhook 串联进工作流。



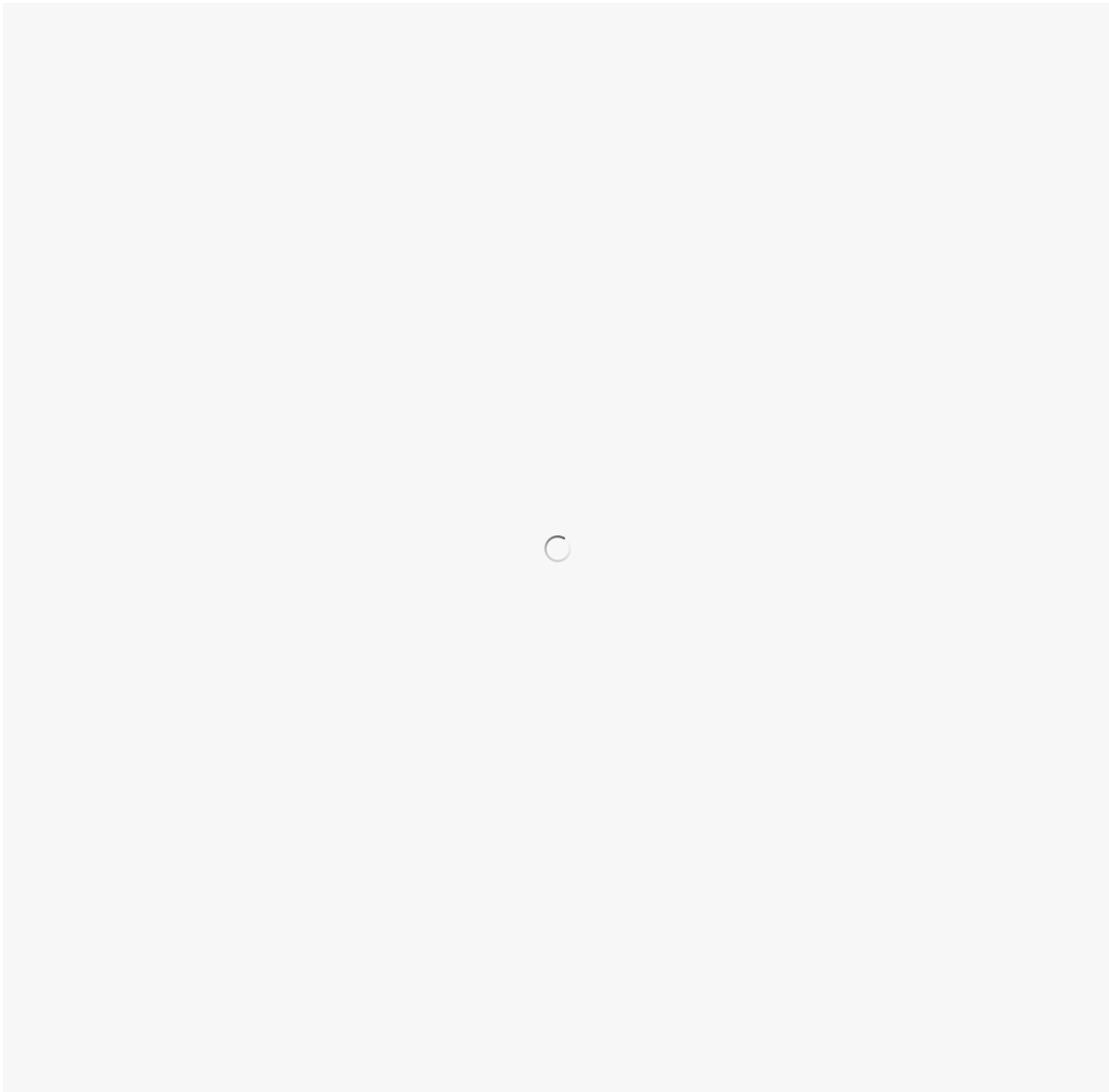
在扣子中搭建工作流

这一部分的目标很明确：**产出一个稳定可用的 workflow API，供后续 TRAE 调用。**

2.1 创建工作流并添加「飞书消息」插件

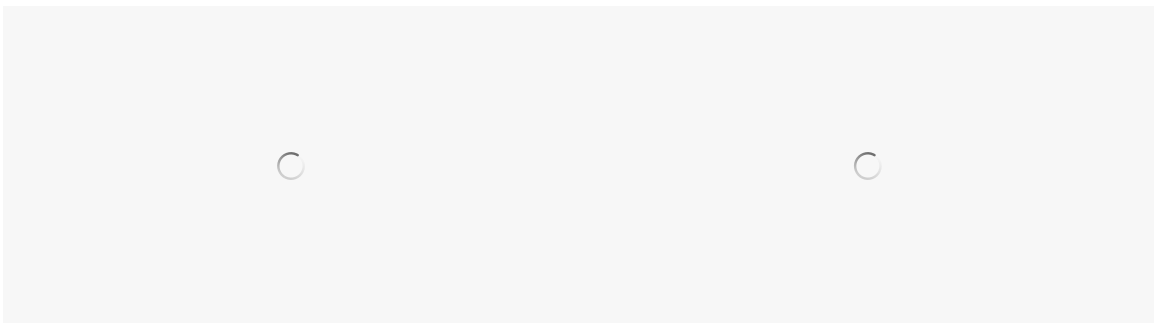
1. 在扣子 workflow 界面中创建一个新的工作流：

- 给它起个名字，例如「飞书群通知」；
- 简单写一下描述（例如“接收文本输入，发送到飞书自定义机器人”）。



2. 在工作流中添加官方插件：「飞书消息」：

- 选择动作：`send_webhook_message`（使用飞书自定义机器人 webhook 发送消息）。



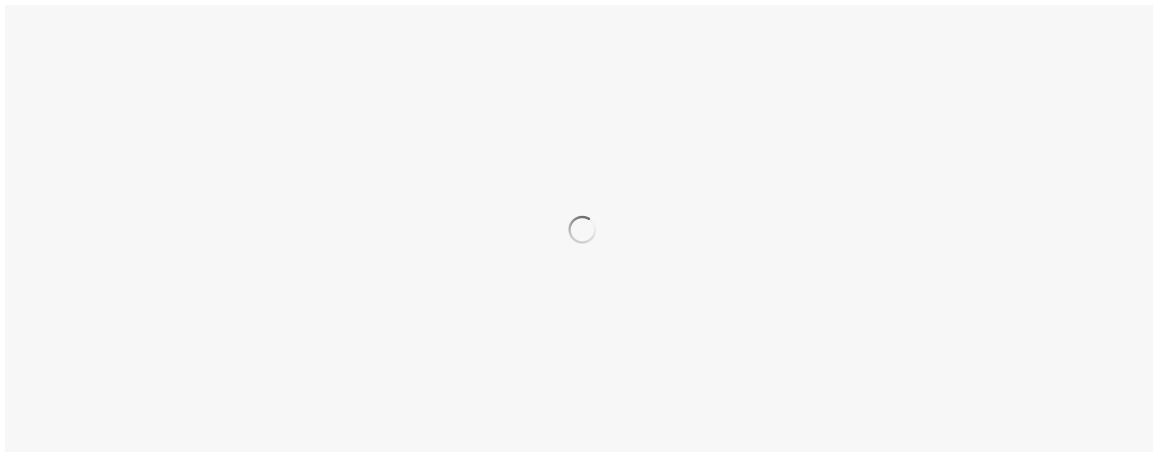
此时工作流包含三个核心节点：

- **开始** 节点：接收一个 `input` 字段；
- **飞书消息** 节点：调用 webhook；
- **结束** 节点：把结果返回出去。

2.2 配置节点与数据流

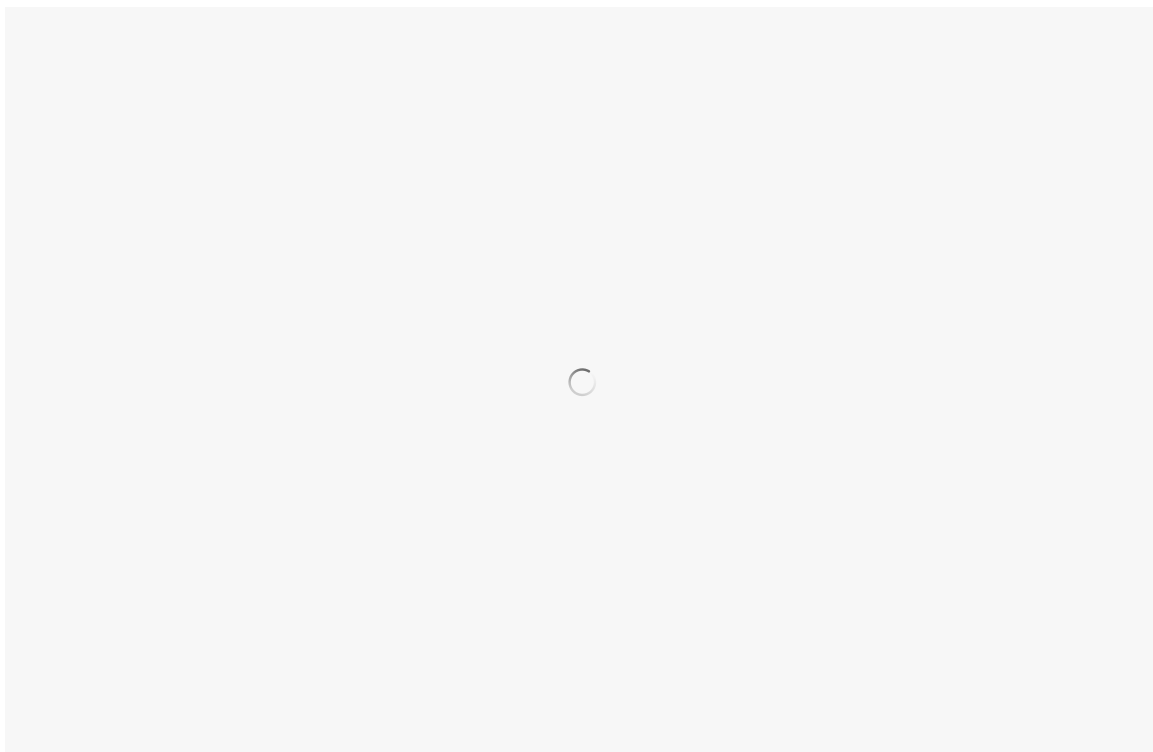
第一步：连接节点

将 **开始** → **飞书消息** → **结束** 三个节点顺序连接起来。



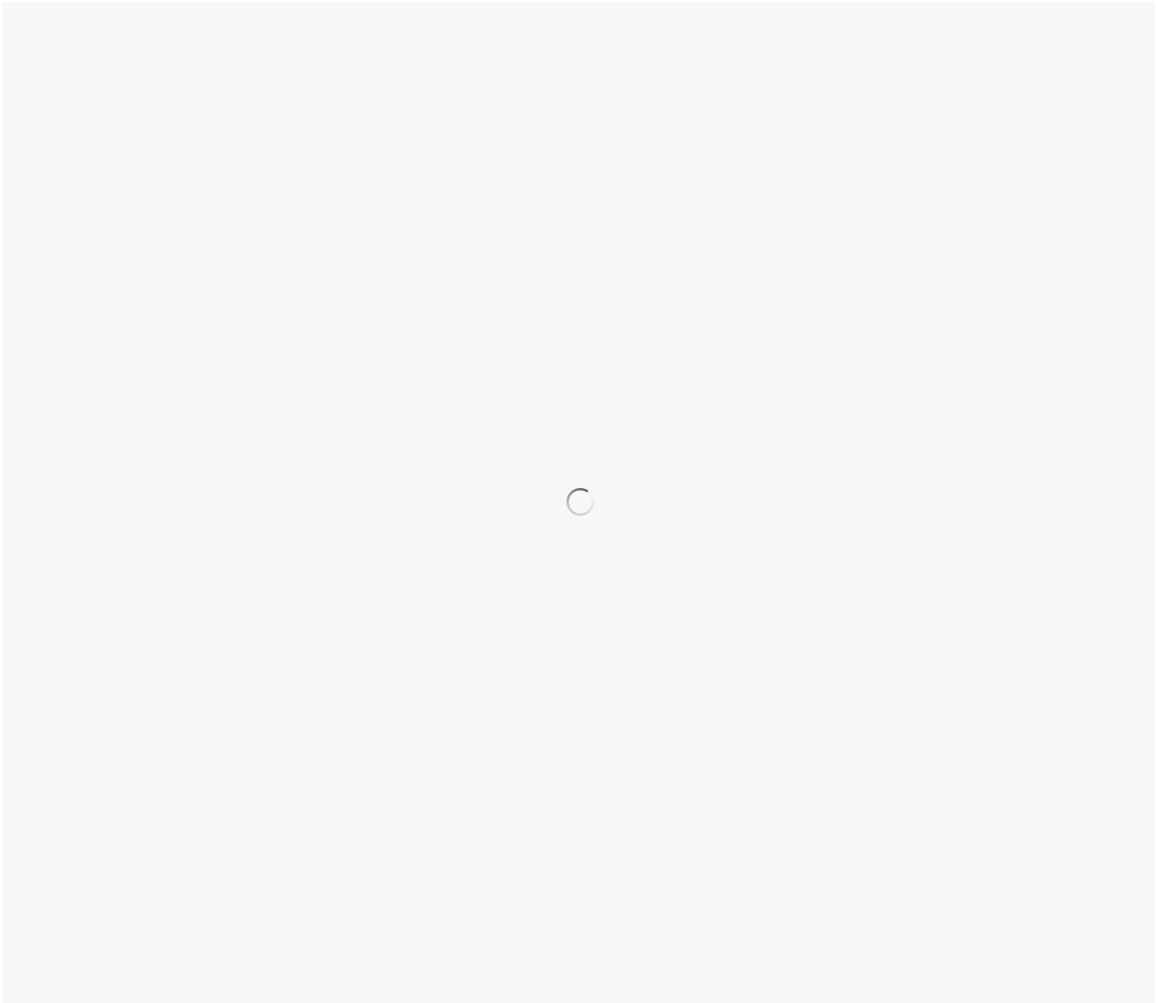
第二步：配置「飞书消息」节点

- **content**：选择 **开始** 节点的 **input** 作为消息内容；
- **msg_type**：填写字符串 **text**（指定发送文本消息）；
- **webhook**：填写前面在飞书群里复制的 Webhook 地址。



第三步：配置「结束」节点输出

- 删除默认输出变量；
- 把「飞书消息」节点的几个关键输出字段（例如 **code**、**msg**、**log_id** 等）逐一添加为结束节点的输出。



2.3 试运行与发布

1. 点击工作流下方的「试运行」：

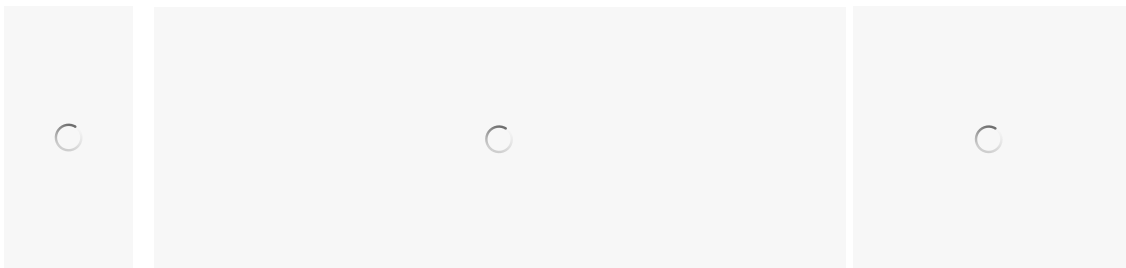
- 在 *input* 中输入任意内容，如「你好」；
- 运行后，确认：

工作流执行成功；

飞书群中机器人成功发出这条消息。

2. 验证无误后，点击右上角「发布」：

- 发布成功后，这个工作流就可以通过 API 访问了。



2.4 在 Playground 中查看 API 调用示例

发布后，在工作流界面的右上角点击「...」，选择「API」：



- 可以看到一段官方提供的 *curl* 示例；
- 包含 *workflow_id*、请求路径、Authorization 头等信息。

这段 *curl* 示例就是我们后面要交给 TRAE 的“原材料”。TRAE 会根据这个示例，自动生成完整的 Service 层代码和测试 UI。



用 TRAE 生成可复用服务

接现在进入本文的核心环节：让 TRAE 替我们完成从 API 文档到可运行代码的全过程。

3.1 给 TRAE 的需求说明

在 TRAE CN SOLO 中，我们可以用自然语言描述需求（伪代码式说明）：

用 vite + React 实现一个可以复用的“工作流服务”：

- 使用扣子的 workflow API：https://api.coze.cn/v1/workflow/stream_run
- 参数包括：
 - token*（Bearer 后面的字符串，由用户输入）；
 - workflow_id*（固定字符串，由用户输入或写死）；
 - input*（要发给飞书机器人的消息内容）；
- 调用成功后，把 workflow 的返回结果展示在 UI 上。
- 请同时：
 - 封装一个独立的 *workflowService*，便于后续复用；
 - 生成一个简单的 React UI 页面，用于测试调用这个 service。



这里有两个关键设计：

- **关注“可复用服务”**：不是写一个一次性 Demo，而是一个可以放进工具箱的 Service；
- **要求生成测试 UI**：方便我们在浏览器里手动输入参数、反复测试。

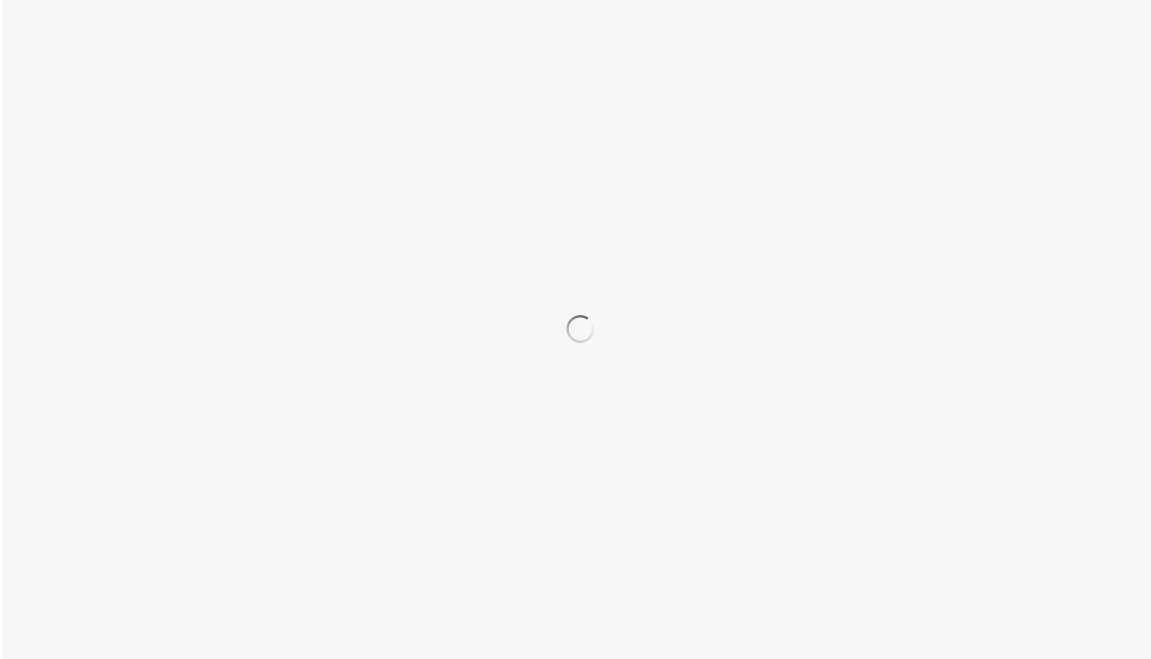
3.2 第一版：能跑，但有问题

TRAE 按照说明，通常会输出两部分代码：

- 一个封装好的 `workflowService`（例如 `workflowService.js` / `workflowService.ts`）；

- 一个简单的 React 页面（带表单和按钮），用来：
 - 输入 *token*、*workflow_id*、*input*；
 - 点击按钮后调用 workflow；
 - 在页面上展示调用结果。

第一版跑起来后，界面是这样的：



1. 在 UI 中填入：
 - Token;
 - Workflow ID；
 - 要发送的文本内容；
2. 点击「运行工作流」；
3. 浏览器发起请求，收到扣子返回的数据。

这时，我们遇到一个问题：



- TRAE 以为 *content* 是一个对象；
- 实际上 *content* 是一个 JSON 字符串，需要再 *JSON.parse* 一次。

结果就是：第一版运行时在控制台报错，解析失败。（可以理解，因为我们没有把输出数据的结构明确告知 AI）

3.3 错误驱动迭代：让 TRAE 自我修复



这一点其实是 TRAE 的“爽点”：

- 在真实开发里，你也不可能一次就完全猜对返回结构；（不用猜，会有详细的返回数据类型的文档）
- 好在我们可以：
 - 把 TRAE 浏览器里报错的 stack trace 复制出来；
 - 再把提示词一起贴给 TRAE；
 - 让它 **根据真实错误和真实数据结构自动修复代码。**

例如你可以这样对 TRAE 说：

刚才你生成的代码在解析返回时出错了，报错信息是：

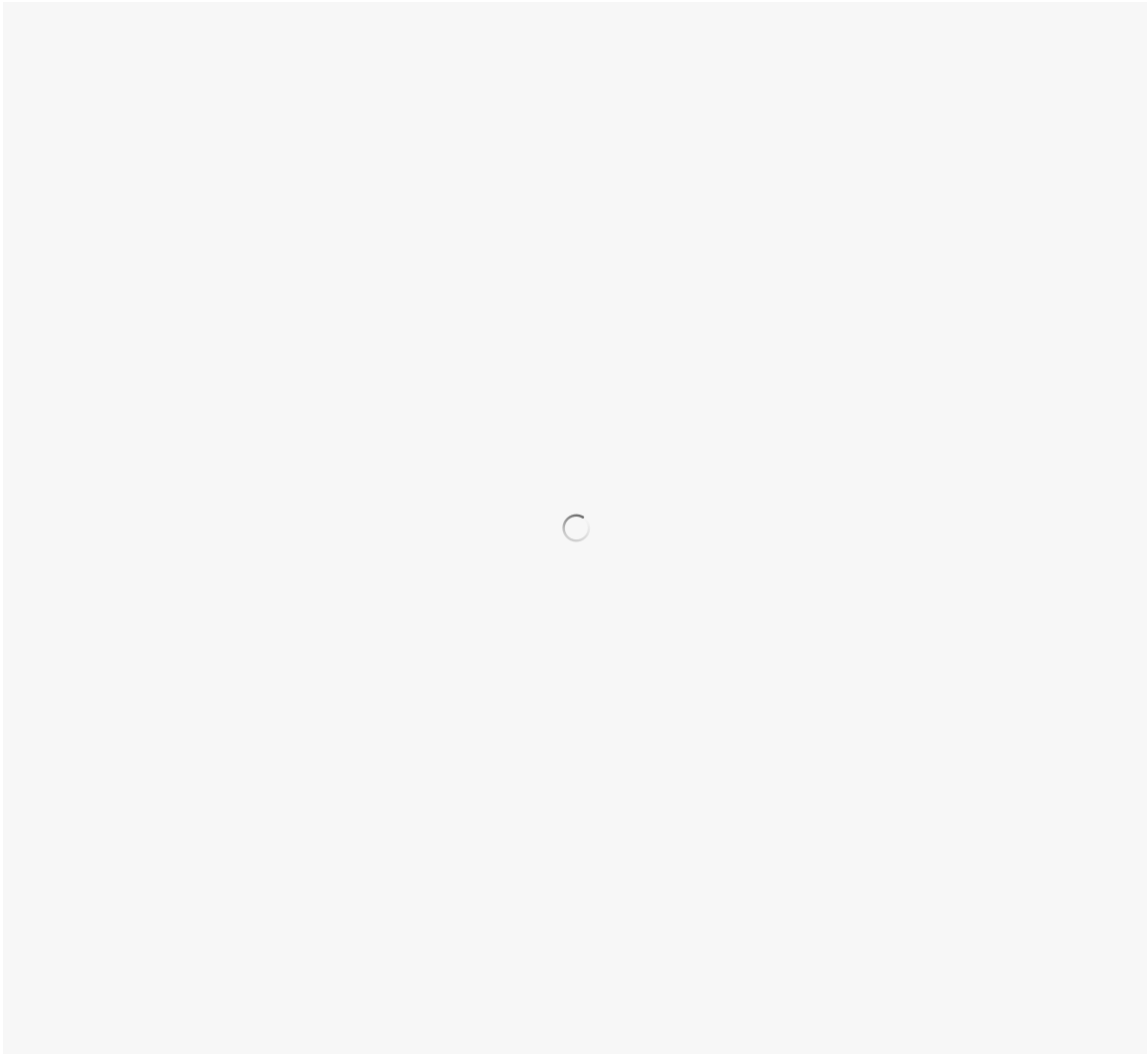
TypeError: Cannot read properties of undefined (reading 'code')

实际返回的完整数据结构如下（贴 JSON）

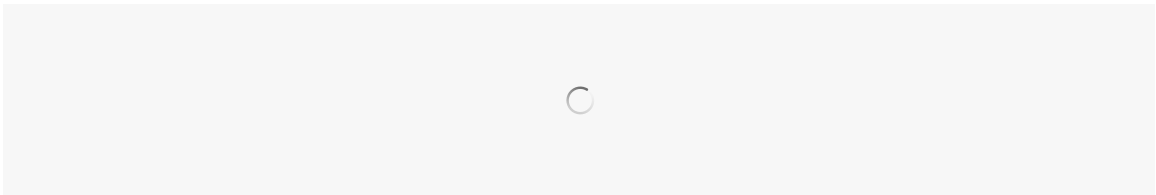
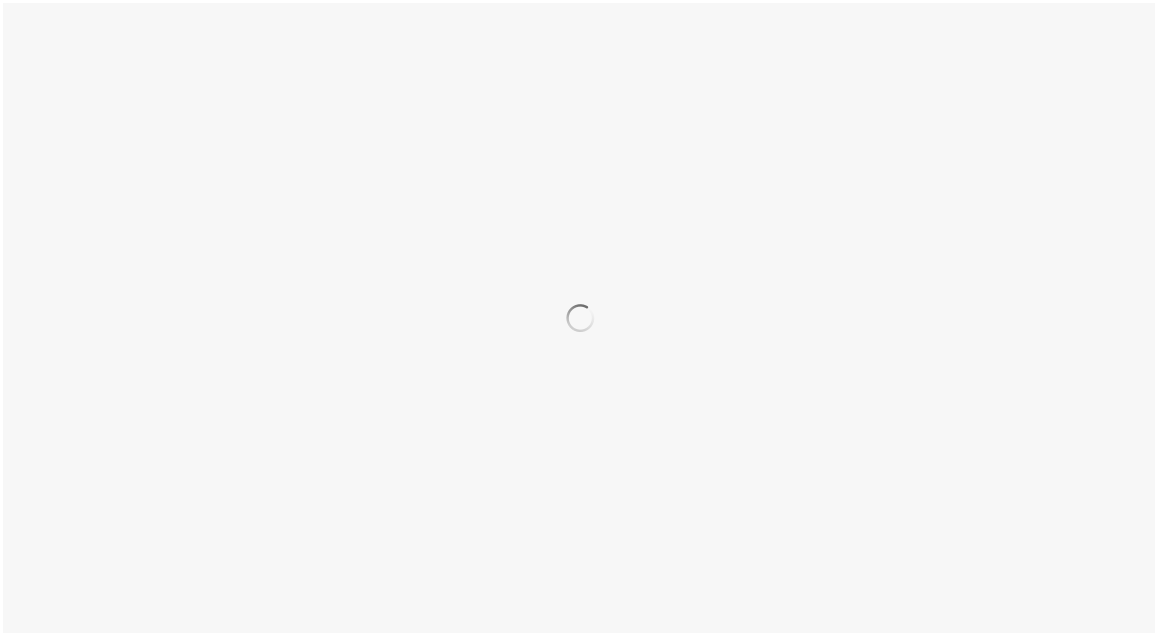
请根据真实结构修正 *workflowService* 和 UI 中的解析逻辑，确保：

- 能正确解析 *content* 字段里的 JSON 字符串；
- 在 UI 上显示 *code* 和 *msg*；
- 遇到错误时给出清晰的提示信息。

TRAE 做出了下面的修改：



再次运行后，只要飞书那边配置无误，就可以看到运行成功了：



3.4 沉淀为工具：workflowService 变成工具箱里的积木

在多轮迭代之后，我们会得到一个相对稳定的服务封装，例如：

一个函数 `runWorkflow({ token, workflowId, input })`：

- 内部封装了：
 - 请求 URL；
 - 请求头（Authorization / Content-Type）；
 - 请求体格式；
 - 返回结果解析和错误处理；
- 返回一个结构化的结果对象（例如 `{ code, msg, raw }`）。

这就是本文想要强调的「工具」：

- **对前端而言：**
 - 任何页面只要引入 workflowService，就能一键调用这个扣子 workflow；
 - UI 部分可以自由组合：按钮、表单、对话框都行。
- **对多端而言：**
 - 这个服务的“调用协议”已经厘清了；
 - 你可以用同样的协议，在 React / Swift / Android / Python 中各写一份同构封装。

TRAE 在这里做了两件关键的事：

- **加速第一次实现：**从 API 文档到可运行 Demo，节省了大量手工编码时间；
- **帮你摸清边界：**通过错误驱动迭代，最终把这个 API 调用“抹平”成一个干净的服务。



小结

回顾一下这整个流程：

1. 我们在扣子里搭建了一个「飞书群通知」工作流，并通过飞书自定义机器人 webhook 把消息发进群；
2. 扣子把这个工作流暴露成一个 HTTP API，附带 curl 示例；
3. 我们把 curl 示例和期望行为交给 TRAE，让它：
 - 自动生成前端 Service (`workflowService`)；
 - 自动生成一个测试 UI 界面；
4. 在运行过程中，遇到返回解析错误，我们把错误和真实返回结构交给 TRAE，让它自动修复；
5. 最终产出的 `workflowService` 被抽离出来，成为工具箱中的一个可复用积木。

TRAE 帮我们“把 API 变成工具”。

从读懂文档、生成代码，到根据实际错误迭代修复，再到沉淀为可复用的服务，这些繁琐却关键的步骤，都是 TRAE 在背后默默完成的。



TRAE.ai 

TRAE, The Real AI Engineer | 字节跳动旗下的AI编程产品，你的专属AI开发工程师... [>](#)
162篇原创内容

公众号

最佳实践 · 目录 

[← 上一篇](#)

四轮迭代，从零到完整游戏：用 SOLO Coder 做俄罗斯方块

[下一篇 →](#)

8 个支持一键导入 TRAE 使用的自定义智能体